AFRILANG Stdlib

std/c/compresshuff

Français. Bibliothèque AFRILANG ‘std/c/compresshuff‘ (niveau complex, catégorie complex). Elle expose 6 fonction(s) : resshuffEncode, resshuffDecode, resshuffRatio, resshuffSize, resshuffCompressed, resshuffRoundTrip.
‘import ”std/c/compresshuff”‘ · ‘use compresshuff‘
- ‘resshuffEncode(data list number) → list number‘ - ‘resshuffDecode(encoded list number) → list number‘ - ‘resshuffRatio(data list number) → number‘ - ‘resshuffSize(data list number) → number‘ - ‘resshuffCompressed(data list number) → number‘ - ‘resshuffRoundTrip(data list number) → bool‘ **English.** AFRILANG library ‘std/c/compresshuff‘

(tier complex, category complex). Exports 6 function(s): resshuffEncode, resshuffDecode, resshuffRatio, resshuffSize, resshuffCompressed, resshuffRoundTrip.
‘import ”std/c/compresshuff”‘ · ‘use compresshuff‘
- ‘resshuffEncode(data list number) → list number‘ - ‘resshuffDecode(encoded list number) → list number‘ - ‘resshuffRatio(data list number) → number‘ - ‘resshuffSize(data list number) → number‘ - ‘resshuffCompressed(data list number) → number‘ - ‘resshuffRoundTrip(data list number) → bool‘

Catégorie / Category	Modules complex (c/)
Niveau / Tier	complex
Fonctions / Functions	6

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/compresshuff' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : resshuffEncode, resshuffDecode, resshuffRatio, resshuffSize, resshuffCompressed, resshuffRoundTrip.

'import "std/c/compresshuff"' · 'use compresshuff'

```
- `resshuffEncode(data list number) → list number`
- `resshuffDecode(encoded list number) → list number`
- `resshuffRatio(data list number) → number`
- `resshuffSize(data list number) → number`
- `resshuffCompressed(data list number) → number`
- `resshuffRoundTrip(data list number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/compresshuff"
use compresshuff
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés — graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 resshuffEncode(data list number) → list•
 resshuffDecode(encoded list number) → list•
 resshuffRatio(data list number) → number•
 resshuffSize(data list number) → number•
 resshuffCompressed(data list number) → number•
 resshuffRoundTrip(data list number) → bool
```

4. Exemples d'utilisation

```
import "std/c/compresshuff"
use compresshuff
```

```
create result = resshuffEncode(/* args */)
say result
```

```
create result = resshuffDecode(/* args */)
say result
```

```
create result = resshuffRatio(/* args */)
say result
```

```
create result = resshuffSize(/* args */)
say result
```

```
create result = resshuffCompressed(/* args */)
say result
```

```
create result = resshuffRoundTrip(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/compresshuff"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module compresshuff

```
function resshuffEncode(data list number) returns list number
end
```

```
function resshuffDecode(encoded list number) returns list number
end
```

```
function resshuffRatio(data list number) returns number
end
```

```
function resshuffSize(data list number) returns number
end
```

```
function resshuffCompressed(data list number) returns number
end
```

```
function resshuffRoundTrip(data list number) returns bool
end
end
```

English

1. Overview

AFRILANG library ‘std/c/compresshuff‘ (tier complex, category complex). Exports 6 function(s): resshuffEncode, resshuffDecode, resshuffRatio, resshuffSize, resshuffCompressed, resshuffRoundTrip.

‘import ”std/c/compresshuff”‘ · ‘use compresshuff‘

```
- `resshuffEncode(data list number) → list number`
- `resshuffDecode(encoded list number) → list number`
- `resshuffRatio(data list number) → number`
- `resshuffSize(data list number) → number`
- `resshuffCompressed(data list number) → number`
- `resshuffRoundTrip(data list number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/compresshuff"
use compresshuff
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 resshuffEncode(data list number) → list•
 resshuffDecode(encoded list number) → list•
 resshuffRatio(data list number) → number•
 resshuffSize(data list number) → number•
 resshuffCompressed(data list number) → number•
 resshuffRoundTrip(data list number) → bool
```

4. Usage examples

```
import "std/c/compresshuff"
use compresshuff
```

```
create result = resshuffEncode(/* args */)
say result
```

```
create result = resshuffDecode(/* args */)
say result
```

```
create result = resshuffRatio(/* args */)
say result
```

```
create result = resshuffSize(/* args */)
say result
```

```
create result = resshuffCompressed(/* args */)
say result
```

```
create result = resshuffRoundTrip(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/compresshuff"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module compresshuff

```
function resshuffEncode(data list number) returns list number
end
```

```
function resshuffDecode(encoded list number) returns list number
end
```

```
function resshuffRatio(data list number) returns number
end
```

```
function resshuffSize(data list number) returns number
end
```

```
function resshuffCompressed(data list number) returns number
end
```

```
function resshuffRoundTrip(data list number) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/compresshuff"`
- Vérification : `afrilang check`
- Documentation : `https://afrilang.dev/stdlib/std/c/compresshuff/`

Source (extrait)

```
module compresshuff

function resshuffEncode(data list number) returns list number
end

function resshuffDecode(encoded list number) returns list number
end

function resshuffRatio(data list number) returns number
end

function resshuffSize(data list number) returns number
end

function resshuffCompressed(data list number) returns number
```

```
end

function resshuffRoundTrip(data list number) returns bool
end
end
```